

```
#
=====
==
```

```
# Q-Copilot Agent
```

```
# Colab 실행용 전체 코드
```

```
#
=====
==
```

```
!pip install -q langgraph scipy pandas numpy matplotlib google-genai peft transformers
accelerate
```

```
import os
```

```
import numpy as np
```

```
import pandas as pd
```

```
import matplotlib.pyplot as plt
```

```
from scipy import stats
```

```
from typing import TypedDict, List, Dict, Any
```

```
from langgraph.graph import StateGraph, START, END
```

```
from peft import LoraConfig, TaskType
```

```
#
=====
==
```

```
# 1. Gemini API 설정
```

```
#
```

```
=====
```

```
USE_LLM = False
```

```
client = None
```

```
try:
```

```
    from google import genai
```

```
    from google.colab import userdata
```

```
    api_key = userdata.get("AlzaSyA6jQ66J9myVxCgH5lHYOtVkOVhWjjVef0")
```

```
    if api_key:
```

```
        client = genai.Client(api_key=api_key)
```

```
        USE_LLM = True
```

```
        print("Gemini API 연결 완료")
```

```
    else:
```

```
        print("GEMINI_API_KEY가 없어 기본 보고서 생성 방식으로 실행합니다.")
```

```
except Exception as e:
```

```
    print("Gemini API를 사용하지 않고 실행합니다.")
```

```
    print(e)
```

```
#
```

```

=====
==

# 2. LoRA Adapter 설정

#
=====
==

prediction_lora_config = LoraConfig(

    r=8,

    lora_alpha=16,

    lora_dropout=0.05,

    task_type=TaskType.CAUSAL_LM

)

control_lora_config = LoraConfig(

    r=8,

    lora_alpha=16,

    lora_dropout=0.05,

    task_type=TaskType.CAUSAL_LM

)

print("Prediction SLM LoRA Adapter 설정 완료")

print("Control SLM LoRA Adapter 설정 완료")

#
=====
==

```

```
# 3. 시연 데이터
```

```
#
```

```
=====
```

```
user_question = """
```

```
Jig 측정값과 계측기 측정값 차이가 있는지 확인하고,  
품질 이슈가 있으면 필요한 조치 방법을 알려줘.
```

```
"""
```

```
jig_data = [10.1, 10.3, 10.2, 10.5, 10.4, 10.6, 10.3, 10.2, 10.5, 10.4]
```

```
meter_data = [10.4, 10.6, 10.5, 10.8, 10.7, 10.9, 10.6, 10.5, 10.8, 10.7]
```

```
df = pd.DataFrame({  
    "Sample": range(1, len(jig_data) + 1),  
    "Jig": jig_data,  
    "Meter": meter_data,  
    "Difference": np.array(meter_data) - np.array(jig_data)  
})
```

```
display(df)
```

```
#
```

```
=====
```

```
# 4. 시연 데이터 그래프
```

```
#
=====
==
```

```
plt.figure(figsize=(9, 4))
plt.plot(df["Sample"], df["Jig"], marker="o", label="Jig")
plt.plot(df["Sample"], df["Meter"], marker="o", label="Meter")
plt.xlabel("Sample")
plt.ylabel("Measured Value")
plt.title("Jig vs Meter Measurement Data")
plt.legend()
plt.grid(True)
plt.show()
```

```
plt.figure(figsize=(9, 4))
plt.bar(df["Sample"], df["Difference"])
plt.axhline(0, linestyle="--")
plt.xlabel("Sample")
plt.ylabel("Meter - Jig")
plt.title("Measurement Difference")
plt.grid(True)
plt.show()
```

```
#
=====
==
```

5. Agent State 정의

#

=====
==

class AgentState(TypedDict):

user_question: str

jig_data: List[float]

meter_data: List[float]

controller_decision: str

prediction_result: Dict[str, Any]

control_result: Dict[str, Any]

final_report: str

#

=====
==

6. Main LLM Controller Node

#

=====
==

def controller_node(state: AgentState) -> AgentState:

question = state["user_question"]

if USE_LLM:

prompt = f"""

너는 제조 품질 분석 Agent의 Main Controller야.

사용자 질문:

{question}

해야 할 일:

1. 어떤 분석이 필요한지 판단
2. 어떤 SLM을 호출해야 하는지 결정
3. 간단한 실행 계획 작성

이 사례는 Jig와 계측기의 측정값 비교 문제야.

같은 샘플을 두 방식으로 측정했으므로 대응표본 t-test가 적절해.

"""

```
response = client.models.generate_content(  
    model="gemini-2.0-flash",  
    contents=prompt  
)
```

```
decision = response.text
```

else:

```
decision = """
```

사용자는 Jig와 계측기 측정값의 차이를 확인하려고 한다.

같은 샘플을 두 방식으로 측정했으므로 대응표본 t-test가 적절하다.

이후 Prediction SLM으로 품질 이슈 여부를 예측하고,
Control SLM으로 현장 조치 방법을 추천한다.

"""

```
state["controller_decision"] = decision
```

```
return state
```

```
#
```

```
=====
```

```
==
```

```
# 7. Prediction SLM Node
```

```
#
```

```
=====
```

```
==
```

```
def prediction_slm_node(state: AgentState) -> AgentState:
```

```
    jig = np.array(state["jig_data"])
```

```
    meter = np.array(state["meter_data"])
```

```
    t_stat, p_value = stats.ttest_rel(jig, meter)
```

```
    mean_jig = np.mean(jig)
```

```
    mean_meter = np.mean(meter)
```

```
    mean_difference = np.mean(meter - jig)
```

```
    if p_value < 0.05:
```



```
issue_prediction = "품질 이슈 가능성 있음"
```

```
risk_level = "High"
```

```
else:
```

```
issue_prediction = "품질 이슈 가능성 낮음"
```

```
risk_level = "Low"
```

```
prediction_result = {
```

```
    "slm_name": "Prediction SLM with LoRA Adapter",
```

```
    "lora_r": prediction_lora_config.r,
```

```
    "lora_alpha": prediction_lora_config.lora_alpha,
```

```
    "test_method": "Paired t-test",
```

```
    "mean_jig": round(float(mean_jig), 4),
```

```
    "mean_meter": round(float(mean_meter), 4),
```

```
    "mean_difference": round(float(mean_difference), 4),
```

```
    "t_statistic": round(float(t_stat), 4),
```

```
    "p_value": round(float(p_value), 8),
```

```
    "risk_level": risk_level,
```

```
    "issue_prediction": issue_prediction
```

```
}
```

```
state["prediction_result"] = prediction_result
```

```
return state
```

```
#
```

```
=====
```

```
==
```

8. Control SLM Node

#

=====
==

```
def control_slm_node(state: AgentState) -> AgentState:
```

```
    prediction = state["prediction_result"]
```

```
    if prediction["p_value"] < 0.05:
```

```
        control_result = {
```

```
            "slm_name": "Control SLM with LoRA Adapter",
```

```
            "lora_r": control_lora_config.r,
```

```
            "lora_alpha": control_lora_config.lora_alpha,
```

```
            "control_decision": "조치 필요",
```

```
            "recommended_actions": [
```

```
                "Jig 보정 상태를 확인한다.",
```

```
                "계측기 검교정 이력을 확인한다.",
```

```
                "동일 샘플을 재측정한다.",
```

```
                "Jig와 계측기 간 상관성 분석을 추가로 수행한다.",
```

```
                "기준 장비를 정하고 측정 기준을 표준화한다."
```

```
            ]
```

```
        }
```

```
    else:
```

```
        control_result = {
```

```
            "slm_name": "Control SLM with LoRA Adapter",
```

```
            "lora_r": control_lora_config.r,
```

```

        "lora_alpha": control_lora_config.lora_alpha,

        "control_decision": "즉시 조치 불필요",

        "recommended_actions": [

            "현재 측정 시스템을 유지한다.",

            "정기적으로 샘플 데이터를 모니터링한다.",

            "추가 데이터 확보 후 재분석한다."

        ]

    }

```

```

state["control_result"] = control_result

return state

```

```

#
=====
==

# 9. Final Report Node

#
=====
==

```

```

def report_node(state: AgentState) -> AgentState:

    prediction = state["prediction_result"]

    control = state["control_result"]

    if USE_LLM:

        prompt = f"""

```

너는 제조 품질 분석 전문가야.

아래 Agent 실행 결과를 바탕으로 발표 시연용 최종 보고서를 작성해줘.

사용자 질문:

```
{state["user_question"]}
```

Main LLM Controller 판단:

```
{state["controller_decision"]}
```

Prediction SLM 결과:

```
{prediction}
```

Control SLM 결과:

```
{control}
```

형식:

1. 분석 목적
2. 선택한 분석 방법
3. 분석 결과
4. 품질 이슈 예측 결과
5. 추천 조치
6. 최종 결론

너무 어렵지 않게 한국어로 작성해줘.

""""

```
response = client.models.generate_content(  
    model="gemini-2.0-flash",  
    contents=prompt  
)
```

```
final_report = response.text
```

```
else:
```

```
    action_text = "\n".join([f"- {action}" for action in  
control["recommended_actions"]])
```

```
    final_report = f"""
```

[Q-Copilot Agent 최종 보고서]

1. 분석 목적

Jig 측정값과 계측기 측정값 사이에 실제 차이가 있는지 확인한다.

2. 선택한 분석 방법

같은 샘플을 Jig와 계측기 두 가지 방식으로 측정했으므로 대응표본 t-test를 사용했다.

3. 분석 결과

- Jig 평균: {prediction["mean_jig"]}
- 계측기 평균: {prediction["mean_meter"]}
- 평균 차이: {prediction["mean_difference"]}

- t 통계량: {prediction["t_statistic"]}

- p-value: {prediction["p_value"]}

4. 품질 이슈 예측 결과

- 예측 결과: {prediction["issue_prediction"]}

- 위험 수준: {prediction["risk_level"]}

5. 추천 조치

{action_text}

6. 최종 결론

p-value가 0.05보다 작으므로 Jig와 계측기 측정값 사이에는 통계적으로 유의한 차이가 있다.

따라서 단순한 측정 오차로 보기 어렵고, Jig 보정 상태와 계측기 검교정 이력을 확인해야 한다.

"""

```
state["final_report"] = final_report
```

```
return state
```

```
#
```

```
=====
```

```
==
```

```
# 10. LangGraph Workflow 구성
```

```
#
```

```
=====
```

```
==
```

```
workflow = StateGraph(AgentState)
```

```
workflow.add_node("main_llm_controller", controller_node)
```

```
workflow.add_node("prediction_slm", prediction_slm_node)
```

```
workflow.add_node("control_slm", control_slm_node)
```

```
workflow.add_node("final_report", report_node)
```

```
workflow.add_edge(START, "main_llm_controller")
```

```
workflow.add_edge("main_llm_controller", "prediction_slm")
```

```
workflow.add_edge("prediction_slm", "control_slm")
```

```
workflow.add_edge("control_slm", "final_report")
```

```
workflow.add_edge("final_report", END)
```

```
app = workflow.compile()
```

```
print("LangGraph Workflow 생성 완료")
```

```
#
```

```
=====
```

```
==
```

```
# 11. Agent 실행
```

```
#
```

```
=====
```

```
==
```

```

initial_state = {
    "user_question": user_question,
    "jig_data": jig_data,
    "meter_data": meter_data,
    "controller_decision": "",
    "prediction_result": {},
    "control_result": {},
    "final_report": ""
}

```

```

result = app.invoke(initial_state)

```

```

#
=====
==

```

```

# 12. 실행 결과 출력

```

```

#
=====
==

```

```

print("\n=====")

```

```

print("Main LLM Controller 판단")

```

```

print("=====")

```

```

print(result["controller_decision"])

```

```

print("\n=====")

```



```
print("Prediction SLM 결과")
```

```
print("=====")
```

```
for key, value in result["prediction_result"].items():
```

```
    print(f"{key}: {value}")
```

```
print("\n=====")
```

```
print("Control SLM 결과")
```

```
print("=====")
```

```
for key, value in result["control_result"].items():
```

```
    print(f"{key}: {value}")
```

```
print("\n=====")
```

```
print("최종 보고서")
```

```
print("=====")
```

```
print(result["final_report"])
```

```
#
```

```
=====
```

```
==
```

```
# 13. 결과 요약 테이블
```

```
#
```

```
=====
```

```
==
```

```
summary_df = pd.DataFrame({
```

```
    "항목": [
```

```

        "분석 방법",
        "Jig 평균",
        "계측기 평균",
        "평균 차이",
        "p-value",
        "품질 이슈 예측",
        "조치 판단"
    ],
    "결과": [
        result["prediction_result"]["test_method"],
        result["prediction_result"]["mean_jig"],
        result["prediction_result"]["mean_meter"],
        result["prediction_result"]["mean_difference"],
        result["prediction_result"]["p_value"],
        result["prediction_result"]["issue_prediction"],
        result["control_result"]["control_decision"]
    ]
})

display(summary_df)

#
=====
==

# 14. 최종 결과 그래프

#

```

```
=====
==
```

```
labels = ["Jig Mean", "Meter Mean"]
```

```
values = [
```

```
    result["prediction_result"]["mean_jig"],
```

```
    result["prediction_result"]["mean_meter"]
```

```
]
```

```
plt.figure(figsize=(6, 4))
```

```
plt.bar(labels, values)
```

```
plt.ylabel("Mean Value")
```

```
plt.title("Mean Comparison: Jig vs Meter")
```

```
plt.grid(axis="y")
```

```
plt.show()
```